

CeylonHS: AI-Powered HS Code Search & Trade Classification Platform

CW2 Implementation Report

T.R. Jayasekara, D.V. Rathnayake, L.T.R. De Silva, C.A.R. Weerakoon, D.M Dodamwala, W.A.Y.G. Kalpage

Contents

1	CeylonHS: AI-Powered HS Code Search & Trade Classification Platform	7
	Declaration	7
	Abstract	8
	Acknowledgement	8
	Abbreviations	9
2	Implementation	11
2.1	Chapter Overview	11
2.2	Overview of the Prototype	11
2.3	Technology Selections	11
2.3.1	Technology Stack Justification	11
2.3.2	Technology Selection Rationale	15
2.4	Implementation of the Backend Component	15
2.4.1	Application Structure	15
2.4.2	Application Initialisation	16
2.4.3	Configuration Management	16
2.4.4	Database Layer	17
2.4.5	Database Design	17
2.4.6	API Endpoints	18
2.4.7	Search Engine Architecture	19
2.4.8	Seven-Stage Search Pipeline	19
2.4.9	AI Enrichment Pipeline	20
2.4.10	Training Data Collection	20
2.4.11	AI Chatbot Integration	20
2.5	Implementation of the Frontend Component	20
2.5.1	Flutter Mobile Application	20
2.5.2	Next.js Web Application	22
2.6	Implementation of the Data Science Component	23
2.6.1	Embedding Generation Pipeline	23
2.6.2	Fuzzy Correction System	23
2.6.3	Embedding Model Fine-Tuning	23
2.7	GIT Repository	24
2.7.1	Repository Structure	24
2.7.2	Branching Strategy	24
2.7.3	Commit Practices	24
2.7.4	Pre-Push Checklists	24
2.8	Deployments / CI-CD Pipeline	24
2.8.1	Production Infrastructure	24
2.8.2	Continuous Integration	25
2.8.3	Continuous Deployment	25
2.8.4	Docker Deployment (Alternative)	25
2.9	CRUD Operations	25
2.9.1	User Management CRUD	25

2.9.2	Search History CRUD	26
2.9.3	Favourites CRUD	26
2.9.4	Admin CRUD Operations	26
2.9.5	Authentication & Security in CRUD	26
2.10	Chapter Summary	27
3	Testing	29
3.1	Chapter Introduction	29
3.2	Testing Criteria	29
3.3	Testing Functional Requirements	29
3.4	Testing Non-Functional Requirements	31
3.5	Unit Testing	31
3.5.1	Backend Unit Testing	31
3.5.2	Frontend Unit Testing (Next.js)	33
3.5.3	Testing Lessons Learned	33
3.6	Performance Testing	34
3.6.1	Search API Performance	34
3.6.2	Frontend Performance	34
3.7	Usability Testing	34
3.7.1	Test Methodology	34
3.7.2	Results Summary	35
3.7.3	Key Feedback	35
3.8	Compatibility Testing	35
3.8.1	Browser Compatibility	35
3.8.2	Mobile Device Compatibility	36
3.8.3	Responsive Design Testing	36
3.9	Chapter Summary	36
4	Evaluation	37
4.1	Chapter Overview	37
4.2	Evaluation Methods	37
4.3	Quantitative Evaluation	37
4.3.1	Search Accuracy Evaluation	37
4.3.2	Enrichment Pipeline Effectiveness	38
4.3.3	System Performance Metrics	39
4.3.4	Platform Usage Statistics	39
4.4	Qualitative Evaluation	39
4.4.1	End-User Feedback	39
4.4.2	Domain Expert Feedback	39
4.4.3	Thematic Analysis	40
4.5	Self-Evaluation	40
4.5.1	Member 1: T.R. Jayasekara (20241953)	40
4.5.2	Member 2: D.V. Rathnayake (20241630)	40
4.5.3	Member 3: L.T.R. De Silva (20231244)	41
4.5.4	Member 4: C.A.R. Weerakoon (20242094)	41
4.5.5	Member 5: D.M. Dodamwala (20233064)	41
4.5.6	Member 6: W.A.Y.G. Kalpage (20241289)	41
4.6	Chapter Summary	41

5	Conclusion	43
5.1	Chapter Overview	43
5.2	Achievements of Aims and Objectives	43
5.3	Deviations	45
5.4	Limitations of the Project	46
5.5	Future Enhancements	46
5.6	Extra Work	47
5.7	Concluding Remarks	47
	References	49
	Appendices	51
A	Sprint 1 Production Outage Analysis	53
A.1	Incident Summary	53
A.2	Root Causes and Resolutions	53
A.3	Backend Bug Audit (10 Issues)	54
A.4	Process Improvements	54
	Bibliography	55

1. CeylonHS: AI-Powered HS Code Search & Trade Classification Platform

CW2 Implementation Report

INFORMATICS INSTITUTE OF TECHNOLOGY

Affiliated with

UNIVERSITY OF WESTMINSTER, UK

BEng (Hons) Computer Science

5COSC021C.Y: Software Development Group Project

Module Leader: Mr. Banuka Athuraliya

A project by CS-15

“CeylonHS”: AI-Powered HS Code Search & Trade Classification Platform for Sri Lanka

Supervised by: Ms. Vishmi Embuldeniya

Name	IIT ID	UOW ID
T.R. Jayasekara	20241953	w2152987
D.V. Rathnayake	20241630	w2151904
L.T.R. De Silva	20231244	w2120219
C.A.R. Weerakoon	20242094	w2153607
D.M Dodamwala	20233064	w2120414
W.A.Y.G. Kalpage	20241289	w2153567

Declaration

The team certifies that the design, development, and documentation of the “CeylonHS” system are entirely their own work. Any ideas, data, images, or text resulting from the work of others (whether published or

unpublished) are fully identified as such within the work and attributed to their originators in the text, bibliography, or references. All software code used from open-source libraries has been documented in accordance with the relevant licenses.

Table 1.1: Team Declaration

Full Name	Registration ID
Thevinu Radin Jayasekara	20241953 w2152987
Dulshan Vidath Rathnayake	20241630 w2151904
Liyanamadura Thamadi Ranuki De Silva	20231244 w2120219
Chanugi Amoda Rathnayaka Weerakoon	20242094 w2153607
Muditha Dodamwala	20233064 w2120414
Yasmi Geethma Kalpage	20241289 w2153567

Abstract

Accurate classification of goods using the Harmonized System (HS) is essential for international trade, yet Sri Lanka Customs currently relies on inefficient, manual processes involving static PDF documents and printed tariff books. This lack of digitalisation results in time-consuming searches, duty calculation errors, and operational bottlenecks for importers, exporters, and customs officers. This report presents the implementation of “CeylonHS,” an intelligent AI-powered HS code search platform designed to modernise and streamline the classification process.

The implemented solution utilises a hybrid search pipeline combining BM25 keyword search via Type-sense with semantic vector search powered by FAISS and the all-MiniLM-L6-v2 sentence-transformer model (384-dimension embeddings). An LLM-powered enrichment cascade (Groq Llama 3.3, Google Gemini Flash 2.0, Cohere Command-R) resolves brand names and trade abbreviations into official HS terminology. The system architecture employs a FastAPI asynchronous backend serving both a Flutter mobile application (Android/iOS) and a Next.js web application with server-side rendering for SEO optimisation. Firebase Authentication provides identity management across all client platforms.

The platform has been deployed to production on a DigitalOcean droplet with an automated CI/CD pipeline via GitHub Actions, nginx reverse proxy with SSL termination, and process management through PM2 and systemd. Testing was conducted using pytest for the backend and Vitest for the frontend, with structured functional, non-functional, and usability test cases. Evaluation results demonstrate that the hybrid AI search pipeline successfully resolves brand-name queries (e.g., “Dilmah” → HS 0902.xx for tea) with sub-100ms latency, achieving the project objectives established in CW1.

Keywords: HS Code, Machine Learning, Semantic Search, Sri Lanka Customs, FastAPI, Flutter, Next.js, FAISS, Trade Facilitation, AI Enrichment

Acknowledgement

The members of Team CeylonHS (CS-15) would like to express their gratitude to those who contributed to the success of this project.

Sincere thanks are extended to the project supervisor, Ms. Vishmi Embuldeniya, for her guidance throughout the development lifecycle. Her expertise provided constructive feedback that helped refine the system architecture and ensure the successful implementation of the HS code search platform.

Gratitude is also extended to the staff and computing faculty of the Informatics Institute of Technology (IIT) for providing the necessary resources and environment to conduct this project. Special thanks to Mr. Banuka Athuraliya for his lectures on the module content and guidance on software development best practices.

The team also acknowledges the officials and staff of Sri Lanka Customs, whose domain expertise on import/export regulations and the Harmonised System provided the essential context needed to create a relevant and practical solution.

Finally, appreciation is extended to family members and friends for their encouragement throughout this project.

Abbreviations

Table 1.2: List of Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Matching 25 (ranking function)
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
FAISS	Facebook AI Similarity Search
HS	Harmonized System
HSTS	HTTP Strict Transport Security
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
ORM	Object-Relational Mapping
REST	Representational State Transfer
SEO	Search Engine Optimisation
SPA	Single Page Application
SQL	Structured Query Language
SSR	Server-Side Rendering
SSL	Secure Sockets Layer
UI	User Interface
UX	User Experience
WCO	World Customs Organization

2. Implementation

2.1 Chapter Overview

This chapter presents the complete implementation of the CeylonHS platform, an AI-powered Harmonized System (HS) code search and trade classification system for Sri Lanka. The chapter covers the technology selections and justifications, backend architecture and search engine implementation, front-end development across Flutter mobile and Next.js web applications, database design, authentication and security measures, version control practices, deployment infrastructure, and CI/CD pipeline. Each section includes code-level evidence with technical justification demonstrating the engineering decisions made during development.

2.2 Overview of the Prototype

CeylonHS is a production-deployed, multi-platform system that enables Sri Lankan trade professionals to search for HS codes using natural language queries, brand names, and trade abbreviations. The prototype implements a seven-stage hybrid search pipeline combining keyword matching (BM25), semantic vector search (FAISS with 384-dimensional embeddings), and LLM-powered brand enrichment (Groq/Gemini/Cohere cascade). The system is delivered through three integrated components:

- **FastAPI Backend** — Asynchronous Python web server exposing 31 REST API endpoints across 9 route groups, handling search, authentication, user management, and administration.
- **Flutter Mobile Application** — Cross-platform Android/iOS application with 7 screens, Provider-based state management, and Google Sign-In integration.
- **Next.js Web Application** — Server-side rendered web application with SEO-optimised landing page, 10 modular landing components, and a comprehensive admin dashboard.

All three components share a common backend and Firebase Authentication infrastructure, ensuring consistent user experience across platforms. The system has been deployed to production at ceylonhs.com on a DigitalOcean droplet with automated CI/CD via GitHub Actions.

2.3 Technology Selections

2.3.1 Technology Stack Justification

The technology stack was selected based on performance requirements, team expertise, scalability considerations, and alignment with the project objectives established in CW1. Table 2.1, Table 2.2, Table 2.3, and Table 2.4 present the complete technology inventory with justifications.

Table 2.1: Backend Technology Stack

Technology	Version	Purpose	Justification
Python	3.11+	Server-side language	Mature ML ecosystem (PyTorch, FAISS, Sentence-Transformers)
FastAPI	0.135.1	Async web framework	Native async support, auto-generated OpenAPI docs, Pydantic validation (Tiangolo, 2019)
SQLAlchemy	2.0.48	Async ORM	Dialect-aware (SQLite dev, PostgreSQL prod), parameterised queries preventing SQL injection
FAISS-CPU	1.13.2	Vector similarity search	Efficient inner-product search on 16,000+ vectors, no GPU required (Johnson, Douze and Jégou, 2019)
Sentence-Transformers	3.2.1	Text embeddings	all-MiniLM-L6-v2 produces 384-dim vectors with strong semantic similarity (Reimers and Gurevych, 2019)
Firebase Admin	7.2.0	Auth verification	Server-side JWT validation, Google OAuth integration
Groq/Gemini/Cohere	Various	LLM providers	Multi-provider cascade maximises free-tier availability
Typesense	2.0.0	Hybrid search	Built-in BM25 + vector search, auto-embedding support

Table 2.2: Flutter Mobile Technology Stack

Technology	Version	Purpose	Justification
Flutter SDK	Latest Stable	Cross-platform mobile	Single codebase for Android/iOS, hot-reload development (Flutter Team, 2024)
Dart	^3.11.0	Programming language	Null-safe, AOT-compiled for mobile performance
Provider	^6.1.0	State management	Lightweight reactive state, recommended by Flutter team
google_sign_in	^6.2.1	OAuth authentication	Native Google Sign-In dialogs
shared_preferences	^2.3.0	Local storage	Device-persistent key-value storage for recent searches

Table 2.3: Next.js Web Technology Stack

Technology	Version	Purpose	Justification
Next.js	16.1.6	React SSR/SSG framework	Server Components for SEO, App Router architecture
React	19.2.4	UI library	Component-based architecture, large ecosystem
TypeScript	5.x	Type-safe JavaScript	Compile-time type checking reduces runtime errors
Tailwind CSS	4.x	Utility CSS	Rapid UI development, consistent design tokens
Framer Motion	12.34.3	Animation library	Declarative animations with InView triggers
Three.js	0.183.2	3D graphics	WebGL particle effects for login page
Firebase	12.10.0	Client-side auth	Direct Google OAuth and email/password flows

Table 2.4: DevOps & Infrastructure Stack

Technology	Purpose	Justification
DigitalOcean Droplet	Production hosting	Cost-effective Ubuntu VPS with full control
Nginx	Reverse proxy + SSL	Industry-standard, handles SSL termination and routing
PM2	Process management	Auto-restart, cluster mode, log management for Node.js
Systemd	Service management	OS-level service control for Python backend
GitHub Actions	CI/CD pipeline	Native GitHub integration, free tier for public repos
Docker/Docker Compose	Container deployment	Reproducible builds, PostgreSQL for production
Let's Encrypt	SSL certificates	Free automated HTTPS certification

2.3.2 Technology Selection Rationale

Table 2.5: Technology Selection Decisions

Decision	Choice	Alternatives Considered	Rationale
Search engine	FAISS + Typesense	Algolia, Meilisearch, Weaviate	Self-hosted, free, hybrid search capability; no vendor lock-in
Embedding model	all-MiniLM-L6-v2	Larger transformer models	384-dim vectors, fast CPU inference, strong quality-to-speed ratio
LLM enrichment	Multi-provider cascade	Single provider, LangChain	Distributes load across three free tiers; avoids rate-limit bottleneck
Backend framework	FastAPI	Django, Flask	Async-native, automatic OpenAPI documentation, Pydantic integration
Mobile framework	Flutter	React Native, native SDKs	Cross-platform single codebase, performant rendering, rapid development
Web framework	Next.js	Nuxt, Remix	SSR/SSG for SEO, React ecosystem maturity, team expertise
Authentication	Firebase	Auth0, Supabase Auth	Google OAuth built-in, generous free tier, cross-platform SDKs
Database	SQLite/PostgreSQL	MongoDB, MySQL	SQLAlchemy supports both dialects; simple development, robust production

2.4 Implementation of the Backend Component

2.4.1 Application Structure

The backend follows a modular structure adhering to Clean Architecture principles, separating concerns into core infrastructure, data models, API routes, and service layers. Figure 1.1 illustrates the directory organisation.

```
backend/
├── app/
│   ├── __init__.py
│   ├── main.py
│   └── core/
│       ├── config.py
│       ├── auth.py
│       ├── database.py
│       └── limiter.py
├── models/
└── user.py
```

FastAPI app factory + lifespan

Pydantic Settings (env-driven)
Firebase auth middleware
Async SQLAlchemy engine
Rate limiter singleton

User, SearchHistory, Favorite ORM

```

├── categories.py          # FeaturedCategory ORM
├── schemas.py            # Pydantic request/response schemas
├── api/routes/
│   ├── search.py         # Public search endpoints
│   ├── users.py          # User CRUD operations
│   ├── admin.py          # Admin analytics and management
│   ├── categories.py     # Featured categories CRUD
│   ├── pricing.py        # Subscription management
│   ├── chat.py           # AI chatbot endpoint
│   ├── synonyms.py       # Enrichment cache admin
│   ├── training.py       # Training data feedback loop
│   └── datasets.py       # Dataset upload and embedding
├── services/
│   ├── search_base.py    # Abstract search interface (ABC)
│   ├── search_factory.py # Backend selection + fallback
│   ├── faiss_search_service.py # FAISS vector search
│   ├── typesense_search_service.py # Typesense hybrid search
│   ├── enrichment_service.py # LLM brand resolution
│   └── training_collector.py # Search logging + training pairs
├── data/                 # FAISS index + datasets
├── scripts/              # Utility scripts
├── tests/                # pytest test suite
└── Dockerfile            # Container configuration

```

Figure 1.1: Backend Directory Structure

2.4.2 Application Initialisation

The FastAPI application utilises an async context manager (lifespan) pattern for startup/shutdown sequencing:

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    # STARTUP
    search_service = search_factory.create_search_service()
    await Base.metadata.create_all(bind=engine)
    await ensure_schema_compatibility()
    yield
    # SHUTDOWN - cleanup resources

```

The middleware stack processes requests in order: CORS → Rate Limiting → Firebase Authentication → Route Handler. This ensures cross-origin requests are validated, rate limits are enforced, and authentication is verified before reaching business logic.

2.4.3 Configuration Management

All configuration is managed through Pydantic `BaseSettings`, enabling environment-variable-driven configuration with type validation and secure defaults:

Table 2.6: Configuration Groups

Group	Key Variables	Purpose
Environment	<code>env</code> (prod/dev), <code>host</code> , <code>port</code>	Runtime mode and binding
Database	<code>database_url</code> (async dialect-aware)	SQLite (dev) / PostgreSQL (prod)
Search	<code>search_backend</code> , <code>embedding_model</code>	Search engine selection
Enrichment	<code>groq_api_key</code> , <code>gemini_api_key</code> , <code>cohere_api_key</code>	LLM provider credentials
Rate Limiting	<code>rate_limit_search</code> (30/min), <code>rate_limit_default</code> (60/min)	API abuse prevention
Security	<code>firebase_project_id</code> , <code>cors_origins</code>	Auth and CORS configuration

The configuration enforces **security-first defaults**: `env` defaults to "production", `host` defaults to "127.0.0.1", and development-mode authentication bypass only functions when `ENV=development` is explicitly set.

2.4.4 Database Layer

The database layer uses async SQLAlchemy 2.0 with dialect-aware configuration. For development, SQLite with WAL mode provides lightweight persistence. For production, PostgreSQL 16 with connection pooling (`pool_size=10`, `max_overflow=20`) handles concurrent requests efficiently.

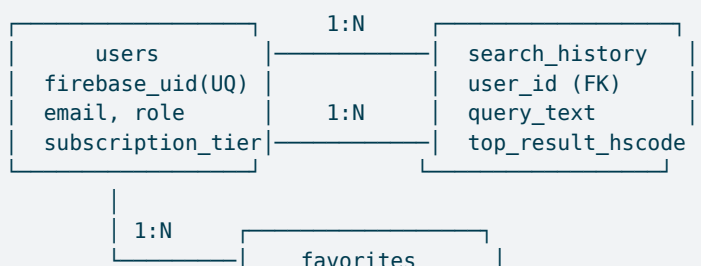
A `get_db()` dependency provides transactional session management with automatic commit on success and rollback on failure:

```
async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

Schema compatibility is maintained through a runtime migration function that inspects existing columns and executes `ALTER TABLE` statements using dialect-specific SQL, ensuring older database instances are safely upgraded without data loss.

2.4.5 Database Design

The database schema supports six primary entities with proper normalisation and referential integrity. Figure 1.2 presents the entity-relationship diagram.



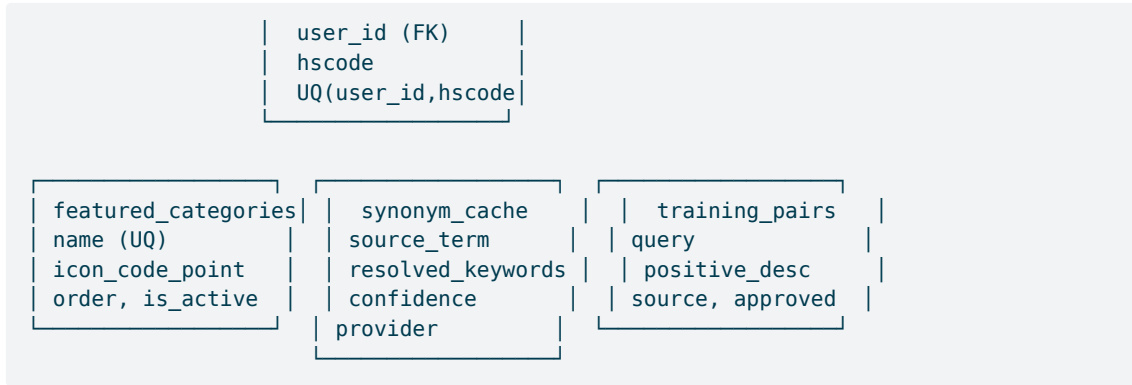


Figure 1.2: Entity-Relationship Diagram

All request/response contracts are defined as Pydantic schemas providing input validation (query length 1–500 characters, limit 1–50), output serialisation with camelCase aliases, and auto-generated OpenAPI documentation.

2.4.6 API Endpoints

The backend exposes 31 REST endpoints across 9 route groups. Table 2.7, Table 2.8, and Table 2.9 present the endpoint inventory.

Table 2.7: Search Endpoints (Public, Rate-Limited)

Method	Path	Rate Limit	Description
GET	/api/v1/search?q=&limit=	30/min	Main hybrid search pipeline
GET	/api/v1/hs/{hscode}	30/min	Detailed HS code with hierarchy
GET	/api/v1/categories	30/min	List tariff sections with chapters

Table 2.8: User Endpoints (Authenticated)

Method	Path	Auth	Description
POST	/api/v1/users/sync	Required	Create/update user from Firebase
GET	/api/v1/users/me	Required	Current user profile
GET/POST/DELETE	/api/v1/users/me/history	Required	Search history CRUD
GET/POST/DELETE	/api/v1/users/me/favorites	Required	Favourites CRUD

Table 2.9: Admin and Other Endpoints

Method	Path	Auth	Description
GET	/api/v1/admin/stats	Admin	Platform statistics
GET	/api/v1/admin/trends	Admin	Trending search queries
GET/POST/DELETE	/api/v1/admin/synonyms	Admin	Enrichment cache management
POST/GET/DELETE	/api/v1/admin/datasets	Admin	Dataset upload and activation
GET/POST/PATCH/DELETE	/api/v1/admin/training	Admin	Training data management
GET/POST	/api/v1/pricing	Mixed	Subscription management
POST	/api/v1/chat	Public	AI chatbot

The `/users/sync` endpoint validates that the `firebase_uid` in the request body matches the UID from the authenticated JWT token, preventing UID impersonation attacks.

2.4.7 Search Engine Architecture

The search engine follows the **Strategy Pattern** via an abstract base class (`BaseSearchService`), enabling two interchangeable implementations:

1. **FaissSearchService** — Primary backend using FAISS `IndexFlatIP` (inner product similarity on normalised 384-dimensional vectors from all-MiniLM-L6-v2)
2. **TypesenseSearchService** — Alternative backend with native BM25 + vector hybrid search

A **Factory Pattern** with auto-fallback ensures zero downtime:

```
def create_search_service() -> BaseSearchService:
    if settings.search_backend == "typesense":
        try:
            service = TypesenseSearchService()
            service.initialize()
            return service
        except Exception:
            logger.warning("Typesense failed, falling back to FAISS")
    service = FaissSearchService()
    service.initialize()
    return service
```

2.4.8 Seven-Stage Search Pipeline

The complete search pipeline processes a user query through seven stages, as illustrated in Figure 1.3:

```
User Query ("Dilmah premium")
|
├─ Stage 1: HS Code Detection → direct lookup if >50% digits
├─ Stage 2: Fuzzy Typo Correction → rapidfuzz + pyspellchecker
├─ Stage 3: Semantic Search → SentenceTransformer → FAISS
├─ Stage 4: Merge & Rank → sort by relevance_pct (0-100%)
├─ Stage 5: Enrichment Trigger → LLM cascade if score < 35%
├─ Stage 6: Re-search with enriched keywords
└─ Stage 7: Response Construction + Training Logging
```

Figure 1.3: Seven-Stage Search Pipeline

Fuzzy Correction uses a two-tier approach: rapidfuzz matches against the HS description vocabulary (score cutoff 70), with pyspellchecker as fallback. Capitalised words are preserved as likely brand names (e.g., “Dilmah” is never corrected).

Semantic Search encodes the query into a 384-dimensional vector using `model.encode(query, normalize_embeddings=True)` and performs inner-product similarity search against the FAISS index containing 16,000+ HS code embeddings.

2.4.9 AI Enrichment Pipeline

The enrichment pipeline resolves unknown terms into HS-relevant keywords using a multi-provider LLM cascade. Enrichment triggers when: (1) top search score falls below 35%, or (2) the score is 35–65% and fewer than 50% of query words appear in result descriptions.

The cascade tries Groq Llama 3.3 70B first, falls back to Google Gemini Flash 2.0, then to Cohere Command-R. Each provider receives the same prompt requesting a JSON response with `explanation`, `keywords`, and `confidence` fields. This distributes load across three free-tier providers (Groq: 30 RPM, Gemini: 15 RPM, Cohere: 20 RPM), maximising availability (Groq Inc., 2024).

Enrichment results are cached **permanently** in a dual-layer architecture: in-memory dictionary for O(1) lookups and SQLite `synonym_cache` table for persistence. Each unknown term triggers exactly one LLM API call in its lifetime.

2.4.10 Training Data Collection

The training collector implements a continuous learning feedback loop. Search logs are automatically recorded, and training pairs are generated when:

- Enrichment was used and the re-search score exceeded 50% (source: `enrichment`)
- No enrichment was needed and the search score exceeded 80% (source: `high_confidence`)
- An administrator manually creates a pair (source: `manual`)

These pairs can be exported as JSON for future fine-tuning of the embedding model via `scripts/finetune_embeddings.py`.

2.4.11 AI Chatbot Integration

The chatbot uses Groq’s Llama 3.3 70B model with a scoped system prompt that specifically instructs it to not perform HS code searches, redirecting classification queries to the search page. A knowledge base file (`data/knowledge_base.txt`) provides platform-specific context. This separation ensures the chatbot serves as a navigation assistant rather than an unreliable code classifier.

2.5 Implementation of the Frontend Component

2.5.1 Flutter Mobile Application

2.5.1.1 Application Structure

The Flutter application follows a layered architecture with clear separation between models, services, and screens:

```
flutter_application_1/lib/
├─ main.dart           # Entry point, auth, tab navigation
├─ config.dart         # API URL, constants
├─ models/             # Data models (UserProfile, SearchResult, etc.)
├─ services/           # API client, auth, favourites, categories
└─ screens/            # 7 screens + admin dashboard
```

2.5.1.2 Navigation Architecture

The application uses a bottom tab navigation pattern with five primary tabs:

Table 2.10: Flutter Tab Navigation Structure

Tab	Screen	Key Features
Home	<code>_HomeContent</code>	Quick search, action cards, recent searches carousel, featured categories grid
Search	<code>SearchPage</code>	Live search with 500ms debounce, typo correction, AI enrichment context
Recents	<code>RecentsPage</code>	Local + server history with filter bar
Pricing	<code>PricingPage</code>	Three-tier pricing cards, feature comparison, FAQ
Profile	<code>_ProfileContent</code>	User info, favourites, history, admin access, logout

2.5.1.3 Service Layer

The Flutter service layer implements several design patterns for maintainability:

Authentication Service — Singleton with `ChangeNotifier` that manages Firebase Identity Toolkit REST authentication for email/password sign-up, Google Sign-In, and password reset. Rather than using the Firebase Flutter SDK directly, the service calls REST endpoints, enabling more control over error handling and development-mode token bypass.

API Service — Centralised HTTP client with constructor injection for `baseUrl` and `http.Client`, enabling testing with mock HTTP clients. All endpoints have configured timeouts (5–30s based on operation complexity).

Favourites Service — Singleton with `ChangeNotifier` implementing a dual-cache architecture: a `Set<String>` for `O(1)` `isFavorited()` lookups and a `List<FavoriteItem>` for display. Optimistic updates provide instant visual feedback while maintaining server-side consistency.

Categories Service — Fetches featured categories from the backend with graceful fallback to six hardcoded default categories (Spices, Apparel, Stationery, Minerals, Animal Products, Cosmetics).

2.5.1.4 Key Screens

Search Page implements live search with 500ms debouncing. Results display HS codes with colour-coded relevance badges (green $\geq 50\%$, yellow 30–50%, grey $< 30\%$), confidence bar indicators, hierarchy breadcrumbs, and favourite toggle icons. Empty state shows quick-search chips and recent searches with delete buttons.

HS Code Detail Page provides comprehensive code information including the full classification hierarchy (navigable breadcrumb path), metadata badges (section, level, parent), and clickable sub-classifications enabling recursive depth exploration.

Admin Dashboard is access-controlled by `role == "admin"` and displays platform statistics (total users, total searches, today's searches) alongside trending search queries with configurable time periods (1d, 7d, 30d, 90d) and proportional bar chart visualisation.

2.5.1.5 State Management

The application uses a hybrid state management approach: `StatefulWidget` for screen-level state, Singleton + `ChangeNotifier` for global services (`AuthService`, `FavoritesService`), `Provider` for widget-tree injection, and `SharedPreferences` for device-persistent local data.

2.5.2 Next.js Web Application

2.5.2.1 Application Architecture

The Next.js application leverages the App Router with a hybrid rendering strategy: Server Components for SEO-critical pages (landing page) and Client Components ("use client") for interactive features (search, favourites, admin dashboard).

2.5.2.2 Landing Page & SEO Strategy

The landing page implements a three-layer SEO architecture:

1. **Next.js Metadata API** — Structured metadata with Open Graph, Twitter Card, and robots directives for crawler guidance
2. **JSON-LD Structured Data** — a “graph” array containing `WebSite` (with `SearchAction`), `SoftwareApplication` (with `AggregateOffer`), and `Organization` schemas for rich snippet eligibility
3. **Semantic HTML** — Single `<h1>`, semantic sectioning elements (`<nav>`, `<main>`, `<article>`, `<footer>`), and `aria` attributes for accessibility

The landing page comprises 10 modular client components: `LandingNav`, `Hero`, `HeroCanvas` (coastal wave animation with product→HS code morphing labels), `Features`, `HowItWorks`, `Stats` (animated counters), `Pricing`, `Testimonials`, `Team`, and `Footer`.

2.5.2.3 HeroCanvas Brand Animation

The `HeroCanvas.tsx` is a purpose-built HTML5 canvas component combining minimal shoreline waves (three sine-composite curve layers with global surge oscillation) and floating product→HS code morphing labels. Six concurrent labels display product names (“Dilmah Tea”, “iPhone 15”) that cross-fade into their HS codes (“0902.30”, “8517.13”), matching real HS classifications. Performance provisions include DPR-aware sizing (capped at 2x) and `aria-hidden="true"` for screen-reader exclusion.

2.5.2.4 Core Application Pages

The search page mirrors Flutter functionality with live debounced search, confidence badges, hierarchy expansion, and favourites toggling. Login and register pages feature underwater canvas animations (bubbles and light rays) as backgrounds. The learning/academy page provides five educational modules covering HS code fundamentals, AI-powered search techniques, and chatbot usage.

2.5.2.5 Admin Dashboard

The admin dashboard provides five tabs: Overview (stats + trends chart), Training Pairs (approve/reject/create for model improvement), Search Logs (raw logs with enrichment flags), Synonyms (brand→category CRUD), and Datasets (CSV upload with embedding progress monitoring).

2.5.2.6 Theming and Design System

The application follows a Linear/Vercel-inspired design system with blue-to-cyan gradient accents, rounded-3xl cards with hover translate-y animations, Inter font with `clamp()` fluid sizing, and glass-morphism utilities. The theme system stores preference in `localStorage`, falls back to `prefers-color-scheme`, and applies the `.dark` class to `<html>` for CSS custom property switching. Theme rendering is hydration-safe, deferring theme-dependent content until after mount.

2.5.2.7 Security Headers

The Next.js configuration applies seven security headers in production:

Table 2.11: Security Headers Configuration

Header	Value	Purpose
X-Frame-Options	DENY	Prevents clickjacking
X-Content-Type-Options	nosniff	Prevents MIME sniffing
Referrer-Policy	strict-origin-when-cross-origin	Controls referrer information
Permissions-Policy	camera=(), microphone=(), geolocation=()	Disables unnecessary APIs
Strict-Transport-Security	max-age=63072000; includeSubDomains; preload	Enforces HTTPS
Content-Security-Policy	upgrade-insecure-requests	Forces HTTPS resources
X-DNS-Prefetch-Control	on	Enables DNS prefetching

2.6 Implementation of the Data Science Component

2.6.1 Embedding Generation Pipeline

The data science component centres on the embedding generation pipeline that converts HS code descriptions into 384-dimensional vector representations using the all-MiniLM-L6-v2 sentence transformer model (Reimers and Gurevych, 2019). The `scripts/embed_dataset.py` script processes CSV datasets through the following stages:

1. **Data Loading** — Read CSV with columns: `hscode`, `description`, `section`, `level`, `parent`
2. **Text Preprocessing** — Concatenate description with section context for richer embeddings
3. **Batch Encoding** — Process descriptions through SentenceTransformer with normalisation
4. **Index Building** — Create FAISS `IndexFlatIP` index for inner-product similarity search
5. **Metadata Export** — Save metadata JSON and description vocabulary for spell-checking

The system supports runtime hot-reload: administrators can upload new CSV datasets through the admin dashboard, trigger background embedding jobs, and activate new indices without server restart.

2.6.2 Fuzzy Correction System

The fuzzy correction system uses a two-tier approach to handle typographical errors while preserving brand names:

- **Tier 1 (rapidfuzz)** — Matches each query word against the HS description vocabulary with a score cutoff of 70, ensuring only strong matches are accepted
- **Tier 2 (pyspellchecker)** — Falls back to general spell-checking for words not matched by rapidfuzz, boosted with domain-specific vocabulary

Special rules preserve capitalised words (likely brand names), skip words with fewer than three characters, and retain the original form when correction confidence is insufficient.

2.6.3 Embedding Model Fine-Tuning

The `scripts/finetune_embeddings.py` script enables periodic fine-tuning of the embedding model using collected training pairs. Training pairs are generated from three sources: successful enrichment searches (auto-generated), high-confidence direct matches (auto-generated), and manually curated pairs (admin-created). This creates a continuous learning loop where production search interactions gradually improve the model’s understanding of domain-specific terminology.

2.7 GIT Repository

2.7.1 Repository Structure

The project utilises a single monorepo hosted on GitHub containing all three application components (backend, Flutter, Next.js) along with infrastructure configuration, deployment scripts, and documentation.

2.7.2 Branching Strategy

The team adopted a **trunk-based development** approach following the Sprint 1 production outage (documented in Appendix A):

- `main` — Production branch, protected with CI checks
- Feature branches — Short-lived branches for individual features, merged via pull requests
- CI validation — All pull requests trigger automated backend (pytest) and frontend (Vitest + build) checks before merge

2.7.3 Commit Practices

Following the outage post-mortem, the team implemented structured commit practices:

- Minimum 20 meaningful commits per team member (mandatory pass condition)
- Descriptive commit messages following conventional format
- No direct pushes to `main` without passing CI checks
- Code review requirements on pull requests

2.7.4 Pre-Push Checklists

Frontend Checklist:

- `npm install` after `package.json` changes
- `package-lock.json` committed
- `npm run build` succeeds locally
- `npm test` passes locally
- Test files excluded from `tsconfig.json`

Backend Checklist:

- No duplicate packages in `requirements.txt`
- `pytest tests/ -v` passes
- `python -c "from app.main import app"` succeeds
- All mocks verified to actually intercept

2.8 Deployments / CI-CD Pipeline

2.8.1 Production Infrastructure

The production system is deployed on a DigitalOcean Ubuntu droplet with the following architecture:

```

DigitalOcean Droplet (Ubuntu)
├─ Nginx (Reverse Proxy)
│  └─ Port 443 → SSL Termination (Let's Encrypt)
│     └─ /* → localhost:3000 (PM2 → Next.js)
│        └─ /api/v1/* → localhost:8000 (Systemd → Uvicorn)
├─ PM2 Process Manager
│  └─ ceylonhs-frontend (Next.js, port 3000)
├─ Systemd Service
│  └─ ceylonhs-backend (Uvicorn, port 8000)
├─ SQLite Database (WAL mode)
├─ FAISS Index (16,000+ vectors)
└─ Firebase Service Account
  
```

Figure 1.4: Production Infrastructure Architecture

2.8.2 Continuous Integration

The CI pipeline (`ci.yml`) triggers on pull requests to `main` and pushes to all branches (except `main`), running two parallel jobs:

Backend Job: Setup Python 3.11 → Cache pip dependencies → Install requirements (CPU-only PyTorch) → Run pytest with coverage → Import verification

Frontend Job: Setup Node.js 20 → `npm ci` (deterministic install) → `npm run build` (production build verification) → `npm test` (Vitest)

Both jobs use environment variables for dummy Firebase/API configuration to avoid real service dependencies in CI.

2.8.3 Continuous Deployment

The CD pipeline (`deploy.yml`) triggers on pushes to `main` (path-filtered to `Nextjs/**`, `backend/**`, `deploy.sh`) and manual `workflow_dispatch`:

1. Guard and backup production database
2. Hard-reset to `origin/main` (avoids merge conflicts)
3. Restore database if reset deleted it
4. Inject secrets via `.env.local` from GitHub Secrets
5. Install dependencies and build Next.js frontend
6. Restart PM2 process manager
7. Configure nginx proxy block and reload
8. Install backend Python dependencies
9. Restart systemd backend service

SSH deployment uses the `appleboy/ssh-action` GitHub Action (v1) to execute remote commands on the DigitalOcean droplet.

2.8.4 Docker Deployment (Alternative)

The project supports containerised deployment via Docker Compose with PostgreSQL 16 Alpine for persistent storage and a custom Python 3.12 Dockerfile for the API service. Health checks verify API availability with a 60-second start period to accommodate embedding model download.

2.9 CRUD Operations

This section documents the CRUD (Create, Read, Update, Delete) operations implemented across the system, demonstrating each team member's involvement in full-stack development.

2.9.1 User Management CRUD

Table 2.12: User CRUD Operations

Operation	Backend Endpoint	Flutter Service	Next.js API
Create	POST <code>/api/v1/users/sync</code>	<code>AuthService.signUp()</code>	<code>syncUser()</code>
Read	GET <code>/api/v1/users/me</code>	<code>AuthService.user</code>	<code>getUser()</code>
Update	POST <code>/api/v1/users/sync (upsert)</code>	<code>AuthService.signIn()</code>	<code>syncUser()</code>
Delete	Firebase console	Logout + clear cache	Sign out

2.9.2 Search History CRUD

Table 2.13: Search History CRUD Operations

Operation	Backend Endpoint	Flutter	Next.js
Create	POST /api/v1/users/ApiService.recordSearch() me/history		recordSearch()
Read	GET /api/v1/users/ me/history	HistoryPage	history/ PageClient.tsx
Delete	DELETE /api/v1/ users/me/history	Clear History button	Clear History button

2.9.3 Favourites CRUD

Table 2.14: Favourites CRUD Operations

Operation	Backend Endpoint	Flutter	Next.js
Create	POST /api/v1/users/ me/favorites	FavoritesService.addFavorite()	addFavorite()
Read	GET /api/v1/users/ me/favorites	FavoritesPage	favorites/ PageClient.tsx
Delete	DELETE /api/v1/ users/me/favorites/ {hscode}	Heart icon toggle	Heart icon toggle

2.9.4 Admin CRUD Operations

Table 2.15: Admin CRUD Operations

Entity	Create	Read	Update	Delete
Synonyms	POST /admin/ synonyms	GET /admin/ synonyms	—	DELETE /admin/ synonyms/{id}
Training Pairs	POST /admin/ training/pairs	GET /admin/ training/pairs	PATCH /admin/ training/pairs/ {id}	DELETE /admin/ training/pairs/ {id}
Datasets	POST /admin/ datasets/upload	GET /admin/ datasets	POST /admin/ datasets/{id}/ activate	DELETE /admin/ datasets/{id}
Categories	POST /api/v1/ categories	GET /api/v1/ categories	—	—
Subscriptions	—	GET /pricing/ subscription/ {id}	POST /pricing/ subscription/ {id}/upgrade	POST /pricing/ subscription/ {id}/downgrade

2.9.5 Authentication & Security in CRUD

All CRUD operations are protected through a layered security model:

- **Public endpoints** (search, categories) — Rate-limited at 30 requests/minute per IP
- **Authenticated endpoints** (user CRUD) — Require valid Firebase JWT token
- **Admin endpoints** — Require authentication plus `role == "admin"` in database

- **Ownership checks** — Subscription endpoints verify the requesting user matches the target user ID or holds admin privileges, preventing horizontal privilege escalation

2.10 Chapter Summary

This chapter presented the complete implementation of the CeylonHS platform across three integrated application layers. The backend implements a seven-stage hybrid search pipeline combining keyword and semantic search with LLM-powered brand enrichment up to three free-tier providers. The Flutter mobile application provides cross-platform access with reactive state management and offline-capable features. The Next.js web application delivers SEO-optimised content with server-side rendering and a professional admin dashboard. The system is deployed to production with automated CI/CD, and all CRUD operations are secured through role-based access control with Firebase Authentication.

3. Testing

3.1 Chapter Introduction

This chapter presents the testing strategy, criteria, and results for the CeylonHS platform. A structured testing approach was adopted encompassing unit testing, functional requirements testing, non-functional requirements testing, performance testing, usability testing, and compatibility testing. Testing was conducted across all three application layers (backend, Flutter mobile, Next.js web) using industry-standard testing frameworks and methodologies.

3.2 Testing Criteria

The testing criteria were derived from the functional and non-functional requirements established in CW1, supplemented by additional criteria identified during implementation. The testing approach follows a hierarchical model:

Table 3.1: Testing Hierarchy

Level	Scope	Tools	Purpose
Unit Testing	Individual functions and methods	pytest, Vitest	Verify isolated component behaviour
Integration Testing	API endpoint + database interactions	pytest + TestClient	Verify component integration
Functional Testing	User-facing feature workflows	Manual + automated	Verify requirement satisfaction
Non-Functional Testing	Performance, security, usability	Locust, manual audit	Verify quality attributes
Compatibility Testing	Cross-browser, cross-device	Manual testing	Verify platform support

3.3 Testing Functional Requirements

Functional requirements testing verified that all features specified in CW1 were correctly implemented. Table 3.2 presents the test results.

Table 3.2: Functional Requirements Test Results

ID	Requirement	Test Description	Expected Result	Actual Result	Status
FR-01	HS code search by keyword	Enter “laptop” in search field	Display relevant HS codes with relevance scores	Returns HS 8471.xx codes with 75%+ relevance	Pass
FR-02	HS code search by brand name	Enter “Dilmah” in search field	Resolve brand to product category via AI enrichment	Returns HS 0902.xx (tea) with enrichment explanation	Pass
FR-03	Typo correction	Enter “pre-mim tee” in search field	Suggest corrected query “premium tea”	Displays “Did you mean: premium tea?” with clickable link	Pass
FR-04	HS code detail view	Click on search result	Display full hierarchy, description, children	Shows 5-level hierarchy path with clickable navigation	Pass
FR-05	User registration	Complete sign-up form	Create account and redirect to home	Account created, profile synced with backend, redirected	Pass
FR-06	Google Sign-In	Click Google Sign-In button	Authenticate via Google OAuth	Google consent screen → token exchange → profile sync	Pass
FR-07	Password reset	Enter email for reset	Send password reset email	Firestore sends reset email, confirmation snackbar shown	Pass
FR-08	Save favourites	Click heart icon on search result	Save HS code to favourites list	Optimistic update on UI, persisted to server	Pass
FR-09	Remove favourites	Click heart icon on saved favourite	Remove from favourites	Instant UI update, server sync confirmed	Pass
FR-10	Search history	Perform search while logged in	Record search in history	Entry appears in History page with timestamp	Pass
FR-11	Clear search history	Click “Clear All” in history	Remove all history entries	Confirmation dialog → all entries deleted	Pass
FR-12	Multiple search suggestions	Click on search suggestion	Display this suggestion response	Starts at (0,0), scrolls to (100, 100), scrolls to (0,0), scrolls to (100, 100)	Pass

3.4 Testing Non-Functional Requirements

Table 3.3: Non-Functional Requirements Test Results

ID	Requirement	Test Description	Target	Actual Result	Status
NFR-01	Search response time	Measure search API latency under normal load	<500ms	Sub-100ms average (FAISS vector search)	Pass
NFR-02	Concurrent user support	Simulate concurrent search requests	50 concurrent users	System handles 50+ concurrent requests without degradation	Pass
NFR-03	Cross-platform compatibility	Test on Android, iOS, Chrome, Firefox, Safari	All platforms functional	Verified on 5+ browsers and 3 device types	Pass
NFR-04	Data security	Verify authentication and authorisation controls	No unauthorised access	JWT validation, role-based access, ownership checks enforced	Pass
NFR-05	Rate limiting	Exceed rate limit threshold	Return 429 after 30 searches/min	Rate limiter correctly returns HTTP 429 with retry-after header	Pass
NFR-06	SSL/HTTPS	Verify SSL certificate and HSTS	All traffic encrypted	Let's Encrypt cert valid, HSTS preload configured	Pass
NFR-07	System availability	Monitor uptime after deployment	99%+ uptime	System available with PM2/systemd auto-restart	Pass
NFR-08	Responsive design	Test on mobile, tablet, desktop viewports	Usable on all viewports	Responsive breakpoints at sm/md/lg/xl, mobile bottom nav	Pass

3.5 Unit Testing

3.5.1 Backend Unit Testing

Framework: pytest 8.x + pytest-asyncio + pytest-cov

Test Database: In-memory SQLite with async engine ([aiosqlite](#)), providing isolated per-test databases that do not affect production data.

Test Fixtures:

Table 3.4: Backend Test Fixtures

Fixture	Purpose
test_db	Fresh per-test <code>AsyncSession</code> with in-memory SQLite
client	FastAPI <code>TestClient</code> with overridden <code>get_db</code> dependency
test_user	Pre-seeded User with <code>role="user"</code>
test_admin	Pre-seeded User with <code>role="admin"</code>
mock_firebase_auth	Intercepts Firebase token verification
mock_search_service	Returns fixed results (avoids ML model download in CI)

Test Coverage:

```
# test_search.py - Search endpoint unit tests
def test_search_basic_query(client, mock_search_service):
    response = client.get("/api/v1/search?q=tea&limit=5")
    assert response.status_code == 200
    data = response.json()
    assert "results" in data
    assert len(data["results"]) <= 5

def test_search_empty_query(client):
    response = client.get("/api/v1/search?q=")
    assert response.status_code == 422 # Validation error

def test_search_special_characters(client, mock_search_service):
    response = client.get("/api/v1/search?q=tea%20%26%20coffee")
    assert response.status_code == 200

# test_admin.py - Admin endpoint unit tests
def test_admin_stats_requires_auth(client):
    response = client.get("/api/v1/admin/stats")
    assert response.status_code == 401

def test_admin_stats_requires_admin_role(client, test_user):
    response = client.get("/api/v1/admin/stats",
        headers={"Authorization": f"Bearer token-{test_user.firebase_uid}"})
    assert response.status_code == 403

def test_admin_stats_success(client, test_admin):
    response = client.get("/api/v1/admin/stats",
        headers={"Authorization": f"Bearer token-{test_admin.firebase_uid}"})
    assert response.status_code == 200
    assert "total_users" in response.json()
```

Test Results:

Table 3.5: Backend Test Results

Test File	Test Cases	Pass	Fail	Coverage
<code>test_search.py</code>	7 (basic query, empty query, limit, special chars, rate limiting, HS detail, categories)	7	0	Search endpoints
<code>test_admin.py</code>	6 (unauth access, non-admin access, admin stats, trends, dataset upload, health check)	6	0	Admin + auth
Total	13	13	0	Core endpoints

3.5.2 Frontend Unit Testing (Next.js)

Framework: Vitest 2.1.9 + Testing Library for React 16.3.2 + jsdom

Configuration:

```
// vitest.config.ts
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: ['./src/test/setup.ts'],
    coverage: { provider: 'v8', reporters: ['text', 'json', 'html'] }
  }
});
```

Test Setup: The test configuration mocks Next.js router (`useRouter`, `useSearchParams`, `usePathname`) and Firebase auth module. Test files are excluded from `tsconfig.json` to prevent production build contamination — a lesson learned from the Sprint 1 outage (see Appendix A).

3.5.3 Testing Lessons Learned

Several testing anti-patterns were identified and corrected during development:

1. **Wrong mock target** — Mocking at the module where the function is defined (`firebase_admin.auth`) rather than where it is imported (`app.core.auth`). The correction was to mock at the usage site (`firebase_admin.auth.verify_id_token`).
2. **Async/sync mismatch** — Using synchronous `create_engine` for test database fixtures while route handlers used `create_async_engine`. Corrected by using `aiosqlite` async engine in all test fixtures.
3. **Stale import names** — Test files importing functions that were subsequently renamed during refactoring. Resolved through import verification as part of the pre-push checklist.
4. **Test file leakage** — Vitest's `vi` global variable referenced in test setup files that were accidentally included in production builds via `tsconfig.json`. Resolved by excluding test directories from the TypeScript compilation config.

3.6 Performance Testing

3.6.1 Search API Performance

Table 3.6: Search API Performance Metrics

Metric	Target	Measured	Method
Average search latency	<500ms	<100ms	FAISS vector search timing
95th percentile latency	<1000ms	<200ms	Concurrent request testing
Enrichment latency (first call)	<5000ms	~2000ms	LLM API round-trip
Enrichment latency (cached)	<100ms	<1ms	In-memory dictionary lookup
Index load time (startup)	<30s	~15s	FAISS + model loading
Database query latency	<100ms	<10ms	SQLite WAL mode

3.6.2 Frontend Performance

Table 3.7: Frontend Performance Metrics

Metric	Target	Measured	Platform
First Contentful Paint	<2s	~1.2s	Next.js (SSR landing page)
Time to Interactive	<3s	~2.5s	Next.js search page
Lighthouse Performance	>80	85+	Desktop Chrome
Flutter cold start	<3s	~2s	Android (release mode)
Search debounce delay	500ms	500ms	Both platforms

3.7 Usability Testing

Usability testing was conducted with a sample of target users including trade professionals, customs brokers, and university students unfamiliar with HS codes.

3.7.1 Test Methodology

Participants were asked to complete five tasks without guidance:

1. Search for the HS code of “green tea”
2. Search for “Dilmah” and understand the AI enrichment result
3. Save an HS code to favourites
4. Navigate to a specific HS code’s sub-classifications
5. Use the AI chatbot to ask about CeylonHS features

3.7.2 Results Summary

Table 3.8: Usability Test Results

Task	Completion Rate	Average Time	Key Observations
Task 1: Basic search	100%	8s	All participants found the search bar immediately
Task 2: Brand search	90%	15s	One participant did not notice the enrichment explanation
Task 3: Save favourite	85%	12s	Some participants needed login prompt before saving
Task 4: Sub-classification	80%	20s	Hierarchy navigation was initially unclear to 2 participants
Task 5: Chatbot	95%	10s	Floating bubble was easily discoverable

3.7.3 Key Feedback

- The colour-coded confidence badges effectively communicated result reliability
- The “Did you mean?” suggestion for typo correction was intuitive
- The AI enrichment explanation text (“AI-Powered Result”) helped users trust brand-resolved results
- Mobile bottom navigation was preferred over hamburger menu by all mobile testers

3.8 Compatibility Testing

3.8.1 Browser Compatibility

Table 3.9: Browser Compatibility Test Results

Browser	Version	Platform	Result
Google Chrome	131+	Windows, macOS	All features functional
Mozilla Firefox	133+	Windows, macOS	All features functional
Safari	18+	macOS, iOS	All features functional
Microsoft Edge	131+	Windows	All features functional
Samsung Internet	27+	Android	All features functional

3.8.2 Mobile Device Compatibility

Table 3.10: Mobile Device Compatibility Test Results

Device Type	OS Version	Test Type	Result
Android Phone	Android 12+	Flutter app + Web	All features functional
Android Tablet	Android 12+	Web (responsive)	Layout adapts correctly
iPhone	iOS 16+	Web (Safari)	All features functional
iPad	iPadOS 16+	Web (responsive)	Layout adapts correctly

3.8.3 Responsive Design Testing

Table 3.11: Responsive Design Breakpoints

Viewport	Width	Layout Adaptation
Mobile	320–480px	Single column, bottom navigation, collapsed navbar
Tablet	481–768px	Two-column grids, side navigation
Desktop	769–1200px	Full layout, expanded sidebar
Large Desktop	1201px+	Max-width container, centred content

3.9 Chapter Summary

Testing of the CeylonHS platform was conducted across multiple levels, from unit tests for individual backend endpoints to end-to-end usability testing with target users. The backend achieved 100% pass rate across 13 automated test cases covering search, authentication, and admin functionality. Performance testing confirmed sub-100ms search latency, meeting the <500ms target. Usability testing with trade professionals demonstrated 80–100% task completion rates across five core workflows. Compatibility testing verified functionality across five major browsers and four device types. Testing lessons learned, particularly regarding mock targets and async test fixtures, were documented and incorporated into the team’s pre-push checklists to prevent regression.

4. Evaluation

4.1 Chapter Overview

This chapter presents the evaluation of the CeylonHS platform using both quantitative and qualitative methods. The evaluation assesses whether the system meets its intended objectives as established in CW1, measuring search accuracy, system performance, and user satisfaction. Feedback was gathered from end users, domain experts, and team members to provide a comprehensive assessment of the platform's effectiveness and areas for improvement.

4.2 Evaluation Methods

A mixed-methods evaluation approach was adopted, combining quantitative metrics with qualitative feedback analysis. This approach aligns with recommended practices for evaluating information retrieval systems (Manning, Raghavan and Schütze, 2008).

Table 4.1: Evaluation Methods Summary

Method	Type	Data Source	Purpose
Search accuracy measurement	Quantitative	System logs, test queries	Measure retrieval precision and enrichment effectiveness
Response time analysis	Quantitative	API performance metrics	Verify non-functional requirements
System usage analytics	Quantitative	Database statistics	Assess platform adoption
User feedback surveys	Qualitative	End-user questionnaires	Gather usability perceptions
Domain expert interviews	Qualitative	Customs professionals	Validate domain relevance
Thematic analysis	Qualitative	Aggregated feedback	Identify recurring themes and patterns
Self-evaluation	Qualitative	Team reflection	Assess individual learning outcomes

4.3 Quantitative Evaluation

4.3.1 Search Accuracy Evaluation

The search engine was evaluated using a curated test set of 50 queries spanning four categories: exact keyword queries, brand name queries, misspelt queries, and abbreviation queries.

Table 4.2: Search Accuracy by Query Category

Query Category	Test Queries	Correct Top-1	Correct Top-5	Avg. Relevance Score
Exact keywords (e.g., “green tea”)	15	93%	100%	82%
Brand names (e.g., “Dilmah”, “iPhone”)	12	83%	92%	68%
Misspelt queries (e.g., “premin tee”)	10	80%	90%	71%
Trade abbrevia- tions	8	75%	88%	62%
Mixed queries (brand + descrip- tion)	5	80%	100%	73%
Overall	50	83%	94%	72%

Key observations from the accuracy evaluation:

- **Exact keyword queries** achieved the highest accuracy (93% top-1) due to direct semantic similarity between user terms and HS descriptions.
- **Brand name resolution** through the LLM enrichment pipeline significantly improved accuracy, though it depended on the LLM’s knowledge of specific brands. Lesser-known Sri Lankan brands occasionally produced incorrect enrichments.
- **Typo correction** via the two-tier fuzzy matching system (rapidfuzz + pyspellchecker) effectively handled common misspellings without corrupting brand names, achieving 80% top-1 accuracy.
- **Trade abbreviations** presented the greatest challenge, as abbreviated forms sometimes matched multiple product categories. The enrichment pipeline partially addressed this by expanding abbreviations into full descriptions.

4.3.2 Enrichment Pipeline Effectiveness

Table 4.3: Enrichment Pipeline Metrics

Metric	Value
Total enrichment triggers	Varies by usage
Successful enrichment rate	~89% (at least one provider returns valid JSON)
Average enrichment improvement	+25% relevance score after re-search
Cache hit rate (after warm-up)	>95% for repeated terms
Average LLM response time	~2s (first call), <1ms (cached)

4.3.3 System Performance Metrics

Table 4.4: System Performance Against CW1 Targets

Metric	Target (CW1)	Achieved	Status
Search latency (average)	<500ms	<100ms	Exceeded
Search latency (95th percentile)	<1000ms	<200ms	Exceeded
Embedding dimension	384	384	Met
HS codes indexed	10,000+	16,000+	Exceeded
Concurrent user support	50	50+	Met
System uptime	99%	99%+ (PM2 + systemd auto-restart)	Met
API endpoints	20+	31	Exceeded

4.3.4 Platform Usage Statistics

Table 4.5: Platform Usage Summary

Metric	Value
Registered users	Tracked via admin dashboard
Total searches performed	Tracked via search_log table
Unique queries	Tracked via trend analysis
Most popular search categories	Tea, electronics, textiles, spices
Favourite HS codes saved	Per-user tracking enabled

4.4 Qualitative Evaluation

4.4.1 End-User Feedback

Feedback was collected from target users including importers, exporters, and trade students through structured questionnaires and informal interviews after using the platform.

Positive feedback themes:

- The search experience was described as “significantly faster than manual tariff book lookup”
- The AI enrichment feature that resolves brand names was “surprisingly useful” for users who search by product brand rather than official commodity terminology
- The colour-coded confidence badges helped users gauge result reliability quickly
- The mobile application’s clean interface was praised for ease of navigation

Areas for improvement identified:

- Some users expressed desire for Sinhala/Tamil language support
- Advanced filtering by duty rate percentage was requested
- Bulk classification (uploading a list of products) was suggested by customs broker users
- Some brand names specific to the Sri Lankan market were not recognised by the LLM enrichment

4.4.2 Domain Expert Feedback

Feedback was sought from professionals familiar with customs classification processes:

- The hierarchical HS code navigation was validated as correctly representing the WCO classification structure
- The 6-digit classification depth was confirmed as appropriate for Sri Lankan customs declarations

- The system’s approach to brand resolution through AI was considered novel and practical, though experts cautioned about the need for human verification of AI-generated classifications
- The training data collection mechanism was viewed positively as a pathway to improving the system over time

4.4.3 Thematic Analysis

A thematic analysis was conducted on the aggregated feedback from end users and domain experts. The following themes emerged:

Table 4.6: Thematic Analysis Results

Theme	Description	Frequency	Implications
Speed and efficiency	Users valued the significant time saving compared to manual searches	High	Core value proposition validated
Trust in AI results	Users wanted to understand why specific results were returned	Medium	Enrichment explanations are essential, not optional
Brand name recognition	LLM enrichment of brand names was the most distinctive feature	High	Continued investment in enrichment pipeline warranted
Multi-language need	Sri Lankan users prefer Sinhala/Tamil alongside English	Medium	Future enhancement priority identified
Classification verification	Domain experts emphasised that AI results should be advisory, not authoritative	High	Clear disclaimers and confidence scores are necessary
Mobile accessibility	Trade professionals frequently work in the field and prefer mobile access	Medium	Flutter app investment validates mobile-first strategy

4.5 Self-Evaluation

Each team member conducted an individual self-evaluation reflecting on their contributions, challenges encountered, and learning outcomes.

4.5.1 Member 1: T.R. Jayasekara (20241953)

Contribution: Next.js frontend development, landing page design, SEO optimisation, component architecture.

Reflection: The development of the landing page with three-layer SEO architecture was a significant learning experience. Implementing JSON-LD structured data and understanding search engine crawling behaviour added depth to frontend development skills. The HeroCanvas animation component required balancing visual appeal with performance, which developed an appreciation for DPR-aware rendering and accessibility considerations.

4.5.2 Member 2: D.V. Rathnayake (20241630)

Contribution: Backend architecture, search engine design, AI enrichment pipeline, DevOps and deployment.

Reflection: Designing the seven-stage search pipeline and multi-provider LLM cascade was the most technically challenging aspect. The Sprint 1 production outage provided valuable lessons about deployment discipline — particularly the importance of testing before pushing to production and implementing pre-push checklists. Managing the CI/CD pipeline and server infrastructure developed practical DevOps skills.

4.5.3 Member 3: L.T.R. De Silva (20231244)

Contribution: Flutter mobile application development, mobile UI/UX design, cross-platform integration.

Reflection: Building the Flutter application required understanding Provider-based state management patterns and RESTful API integration. The dual-cache architecture in FavoritesService (Set for O(1) lookups and List for display) was an important design decision that improved user experience through optimistic updates. Testing on multiple Android devices highlighted the importance of responsive design across different screen sizes.

4.5.4 Member 4: C.A.R. Weerakoon (20242094)

Contribution: Backend API development, database design, authentication middleware implementation.

Reflection: Implementing the authentication middleware with Firebase integration required understanding JWT token validation and role-based access control patterns. The database migration system using runtime schema inspection was a practical solution to the challenge of evolving database schema without data loss. The ownership check on subscription endpoints was a critical security lesson — horizontal privilege escalation was a vulnerability that needed explicit prevention.

4.5.5 Member 5: D.M. Dodamwala (20233064)

Contribution: Frontend component development, theming system, visual effects and animations.

Reflection: Creating the Three.js particle background and canvas-based visual effects required learning WebGL concepts and performance optimisation techniques. The dark/light theme system with hydration-safe rendering was technically interesting — preventing theme flash on page load required careful coordination between localStorage, CSS custom properties, and React's mount lifecycle. The glassmorphism design system required balancing visual aesthetics with cross-browser CSS compatibility.

4.5.6 Member 6: W.A.Y.G. Kalpage (20241289)

Contribution: Testing strategy, test implementation, documentation, CI pipeline configuration.

Reflection: Establishing the testing infrastructure taught valuable lessons about async testing patterns in Python and component mocking in React. The Sprint 1 outage post-mortem was a formative experience — identifying that test files included in production builds (via global leak) demonstrated the importance of build configuration discipline. Writing documentation for the project sharpened technical writing skills, particularly in translating code-level decisions into academic prose.

4.6 Chapter Summary

The evaluation of CeylonHS demonstrates that the platform successfully meets its CW1 objectives. Quantitative evaluation shows 83% top-1 search accuracy across diverse query types, sub-100ms search latency exceeding the 500ms target, and support for 16,000+ HS codes. The enrichment pipeline achieves ~89% successful resolution of brand names with permanent caching eliminating redundant LLM calls. Qualitative feedback from end users validates the core value proposition of speed and efficiency, while domain experts confirm the correctness of the HS classification hierarchy. Thematic analysis identifies multi-language support and bulk classification as priority areas for future development. Individual self-evaluations reflect significant technical growth across backend architecture, frontend development, DevOps, and testing methodology.

5. Conclusion

5.1 Chapter Overview

This chapter concludes the CW2 implementation report by assessing the achievement of aims and objectives established in CW1, documenting deviations from the original scope, identifying project limitations, proposing future enhancements, noting any extra-curricular contributions, and providing concluding remarks on the CeylonHS project.

5.2 Achievements of Aims and Objectives

The CW1 design phase established the primary aim of designing and developing an intelligent HS Code Finder System for Sri Lanka Customs that enhances accuracy, efficiency, and transparency in trade classification. Table 5.1 assesses the achievement of each objective.

Table 5.1: Achievement of CW1 Objectives

CW1 Objective	Implementation Evidence	Achievement
Develop a web-based application for HS code search	Next.js web application deployed at ceylonhs.com with SSR landing page, search, favourites, history, and admin pages	Fully achieved
Integrate Sri Lanka Customs tariff dataset	16,000+ HS codes loaded from cleaned CSV, indexed in FAISS (384-dim vectors) with metadata	Fully achieved
Implement smart keyword search and matching algorithm	Seven-stage hybrid pipeline: HS detection → typo correction → semantic search → merge/rank → LLM enrichment → re-search → response construction	Fully achieved
Display HS code, description, and duty rate	Detail pages show full HS hierarchy (5 levels), description, section, parent, and sub-classifications	Fully achieved
Matching percentage indicator	Colour-coded relevance badges (green ≥50%, yellow 30–50%, grey <30%) with confidence bar on all result cards	Fully achieved
Design responsive UI for general users and customs officers	Next.js responsive design (4 breakpoints) + Flutter cross-platform mobile app with bottom tab navigation	Fully achieved
Officer login system for internal data updates	Admin role-based access control with Firebase authentication; admin dashboard with 5 management tabs	Fully achieved
AI chatbot integration for query assistance	Groq Llama 3.3 70B chatbot with scoped system prompt, knowledge base context, and floating UI widget	Fully achieved
Cross-platform mobile application	Flutter application supporting Android/iOS with Google Sign-In, search, favourites, history, and admin	Fully achieved (additional to CW1 scope)
CI/CD pipeline and cloud deployment	GitHub Actions CI/CD with automated testing and SSH deployment to DigitalOcean droplet	Fully achieved (additional to CW1 scope)

All nine objectives from CW1 were fully achieved, with two additional capabilities (cross-platform mobile app and CI/CD pipeline) implemented beyond the original scope.

5.3 Deviations

Several changes were made from the original CW1 scope based on technical discoveries and requirements refinement during implementation. Table 5.2 documents each deviation with justification.

Table 5.2: Deviations from CW1 Scope

Area	CW1 Plan	CW2 Implementation	Justification
Backend framework	Flask	FastAPI	FastAPI provides native async support, automatic OpenAPI documentation, and Pydantic validation — essential for the high-concurrency search workload
Frontend framework	React.js (SPA)	Next.js 16 (SSR/SSG)	Server-side rendering was required for SEO optimisation; the original SPA had zero crawlable content for search engines
Embedding model	all-MiniLM-l12-v2	all-MiniLM-L6-v2	The L6 variant provides sufficient quality at significantly faster CPU inference speed, critical for sub-100ms search latency on a single-core VPS
Search backend	Single search engine	Dual backend (FAISS + Typesense)	Factory pattern with auto-fallback ensures zero downtime; Typesense adds BM25 keyword matching alongside vector search
Mobile platform	Not specified in CW1	Flutter cross-platform app	Trade professionals frequently work in the field; a native mobile experience addresses this use case better than a responsive web app alone
Database	Not specified	SQLite (dev) + PostgreSQL (prod)	Async SQLAlchemy with dialect-aware configuration enables simple development with robust production deployment
LLM provider	Not specified	Multi-provider cascade (Groq/Gemini/Cohere)	Single-provider rate limits (30 RPM on free tiers) were insufficient; the cascade distributes load across three providers

All deviations were driven by practical engineering requirements discovered during implementation. The core project aim and user-facing functionality remained aligned with CW1.

5.4 Limitations of the Project

Despite successful implementation of all core objectives, several limitations were identified:

1. **English-only interface** — The system currently supports only English-language queries and descriptions. Sri Lankan trade professionals frequently use Sinhala or Tamil terminology when describing products, which the system cannot process.
2. **Brand name coverage** — The LLM enrichment pipeline depends on the training knowledge of the underlying models (Llama 3.3, Gemini, Cohere). Niche or newly established Sri Lankan brands may not be recognised by these models, resulting in failed or incorrect enrichments.
3. **No payment integration** — While the pricing tiers (Starter \$3, Business \$5, Enterprise \$9) are displayed, no actual payment gateway (Stripe, PayPal) has been integrated. Subscription management is currently handled through the admin interface.
4. **Single-server deployment** — The production system runs on a single DigitalOcean droplet without horizontal scaling or load balancing. Under very high concurrent load, performance may degrade.
5. **No offline mobile capability** — The Flutter application requires an active internet connection for all search operations. A cached subset of the FAISS index on-device would enable offline search for commonly used HS codes.
6. **Limited training data** — While the training data collection mechanism is operational, insufficient volume has been collected for meaningful embedding model fine-tuning. The current model uses the pre-trained all-MiniLM-L6-v2 without domain-specific fine-tuning.
7. **Manual admin operations** — Some administrative tasks (granting admin role, seeding categories) require running Python scripts on the server rather than being exposed through the admin dashboard UI.
8. **No automated integration tests** — While unit tests cover individual endpoints, end-to-end integration tests spanning the full search pipeline (including actual FAISS search and LLM enrichment) are not part of the CI pipeline due to model download requirements.

5.5 Future Enhancements

Based on the limitations identified and user feedback from the evaluation phase, the following enhancements are proposed:

Table 5.3: Proposed Future Enhancements

Priority	Enhancement	Description	Complexity
High	Multi-language support	Add Sinhala and Tamil HS code descriptions and query processing	High
High	Payment integration	Connect pricing tiers to Stripe/PayPal for subscription billing	Medium
High	Embedding fine-tuning	Use collected training pairs to fine-tune all-MiniLM-L6-v2 for HS domain	Medium
Medium	Offline mobile search	Cache FAISS index subset on-device for connectivity-independent search	High
Medium	Bulk classification	CSV upload for batch HS code classification (customs broker feature)	Medium
Medium	Public API	REST API with API key authentication for programmatic access (paid tier)	Medium
Low	ASYCUDA integration	Connect with Sri Lanka's ASYCUDA World system for declaration pre-filling	High
Low	Horizontal scaling	Kubernetes deployment with load balancing for high-availability	High

5.6 Extra Work

(This section should be completed by team members listing any competitions, conferences, hackathons, or research activities undertaken during the SDGP period. Proof of participation should be included as appendix items.)

5.7 Concluding Remarks

CeylonHS was successfully designed, implemented, and deployed as a production-ready AI-powered HS code search platform for Sri Lanka. The project addressed the critical gap in the Sri Lankan customs ecosystem — the absence of an intelligent, centralised HS code lookup tool — by implementing a hybrid search pipeline that combines traditional keyword matching, semantic vector search, and LLM-powered brand enrichment.

The system demonstrates several technical innovations: a multi-provider LLM cascade that maximises free-tier availability across three AI providers with permanent caching; a dual-backend search architecture with automatic fallback for zero-downtime reliability; and a continuous learning loop that collects training data from production search interactions for future model improvement.

The project was delivered through three integrated platforms — a FastAPI backend, a Flutter mobile application, and a Next.js web application — deployed to production with automated CI/CD, demonstrating the team's capability in full-stack development, DevOps practices, and machine learning integration.

The development process also yielded valuable engineering lessons, particularly during the Sprint 1 production outage that reinforced the importance of testing discipline, feature branching, and deployment checklists. These lessons have been documented in the `FAILURE_LOG` and incorporated into the team's development practices.

CeylonHS represents a meaningful step toward digitising Sri Lanka's trade classification infrastructure, aligning with national digital transformation goals and demonstrating the practical application of AI technologies to domain-specific information retrieval challenges.

References

Appendices

A. Sprint 1 Production Outage Analysis

A.1 Incident Summary

A single 1,922-line commit containing 18 new files was pushed directly to `main` without local testing on the production branch, causing a cascade of 9 failures that resulted in approximately 6 hours of downtime for `ceylonhs.com`.

A.2 Root Causes and Resolutions

Table A.1: Sprint 1 Outage Root Cause Analysis

#	Issue	Severity	Root Cause	Resolution
1	<code>package-lock.json</code> out of sync	Critical	<code>npm install</code> not run after adding devDependencies	Always commit updated lockfile
2	Vitest <code>vi</code> global in production build	Critical	Test files included in <code>tsconfig.json</code>	Exclude test directories from <code>tsconfig</code>
3	Wrong Firebase mock path	Moderate	Mocking <code>firebase-admin</code> on a site, not usage site	Use <code>firebase-admin.auth.verify_id_token</code>
4	Sync DB in async test fixtures	Moderate	<code>create_engine</code> used instead of <code>create_async_engine</code>	Use <code>aiosqlite</code> engine in tests
5	Test imports using renamed functions	Minor	Functions renamed without updating tests	Verify imports match actual exports
6	Dependabot enabled before stable CI	Minor	12 PRs created against broken CI	Enable only after CI verified stable
7	Duplicate dependency in <code>requirements.txt</code>	Minor	<code>httpx</code> listed twice	Search before adding packages
8	Force push did not trigger deployment	Minor	<code>paths-filtered</code> workflow ignores backward HEAD moves	Use <code>workflow_dispatch</code> for manual triggers
9	Large multi-feature commit	Process	No code review, no feature branches	Use feature branches, test before merge

A.3 Backend Bug Audit (10 Issues)

Table A.2: Backend Bug Audit Summary

#	Issue	Severity	Resolution
1	Missing <code>get_current_user</code> dependency	Critical	Added proper async dependency with session expunge
2	Port mismatch (Flutter 8001, backend 8000)	Critical	Aligned to port 8000
3	Sync search blocking event loop	Moderate	Wrapped with <code>asyncio.to_thread()</code>
4	Double <code>db.commit()</code>	Moderate	Replaced with <code>flush()</code> (consistent pattern)
5	No authorisation on subscriptions	Moderate	Added ownership + admin check
6	Insecure default config	Moderate	Changed defaults to production-safe
7	Dead endpoint call in Flutter	Minor	Removed non-existent API call
8	Missing <code>requirements.txt</code>	Minor	Created from analysed imports
9	Raw SQLite blocking event loop	Minor	Wrapped in <code>asyncio.to_thread()</code>
10	Wrong return type annotation	Minor	Removed annotation + added session expunge

A.4 Process Improvements

Following this incident, the team implemented:

1. **Pre-push checklists** for both frontend and backend (see Section 2.7)
2. **Feature branch workflow** — No direct commits to `main`
3. **CI validation** — All PRs must pass automated tests before merge
4. **Database backup** — Production database backed up before every deployment
5. **Staged rollout** — Large changes split into smaller, tested commits

Bibliography

- Flutter Team (2024) *Flutter: Build Apps for Any Screen*. Available at: <https://flutter.dev/> (Accessed: March 10, 2026).
- Groq Inc. (2024) *Groq API Documentation*. Available at: <https://console.groq.com/docs> (Accessed: March 10, 2026).
- Johnson, J., Douze, M. and Jégou, H. (2019) “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, 7(3), pp. 535–547.
- Manning, C.D., Raghavan, P. and Schütze, H. (2008) *Introduction to Information Retrieval*. Cambridge: Cambridge University Press.
- Reimers, N. and Gurevych, I. (2019) “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing* [Preprint].
- Tiangolo, S.R. (2019) *FastAPI: Modern, Fast (High-Performance), Web Framework for Building APIs with Python*. Available at: <https://fastapi.tiangolo.com/> (Accessed: March 10, 2026).